

Software Architecture and Design Specification

Project: Collaborative Study Material and Quiz Platform

Version: 1.0

Authors: Sadanand (PES2UG24CS423), Sagar K (PES2UG24CS424), Sahana K S (PES2UG24CS425), Sahana M S (PES2UG24CS426)

Team No: 13

Date: 03-10-2026

Status: Draft

Revision History

Version	Date	Author	Change Summary
1.0	03-10-2026	Team 13	Initial SAD draft

Approvals

Role	Name	Signature / Date
Course Coordinator	Prof. Asha Kumari	
Team Lead	Sadanand	Sadanand

1. Introduction

1.1 Purpose

This document specifies the software architecture and design of the Collaborative Study Material and Quiz Platform, translating the requirements defined in the SRS v1.0 into a concrete system structure, component design, and interface specifications to guide implementation and testing.

1.2 Scope

Covers the platform's client-server architecture, core backend modules (authentication, study material, quiz management, question bank, collaboration), the Python analytics microservice, database design, API design, security architecture, and UX considerations. Deployment infrastructure provisioning and third-party service internals are outside this document's scope.

1.3 Audience

Developers and the project team, QA engineers, security auditors/evaluators, course instructors, and future maintainers of the platform.

1.4 Definitions

MERN (MongoDB, Express.js, React.js, Node.js), JWT (JSON Web Token), RBAC (Role-Based Access Control), API (Application Programming Interface), REST (Representational State Transfer), ODM (Object Document Mapper), STRIDE (Spoofing, Tampering, Repudiation, Information Disclosure, Denial of Service, Elevation of Privilege), CI/CD (Continuous Integration / Continuous Deployment), TLS (Transport Layer Security).

2. Document Overview

2.1 How to use this document

Provides the architectural deliverables for the mini-project: a component diagram, component descriptions, the chosen architecture pattern and rationale, technology stack, risk analysis, requirements traceability, threat modeling, UML sequence diagrams, API design, and UX/error-handling guidance.

2.2 Related Documents

Collaborative Study Material and Quiz Platform SRS v1.0; Software Test Plan (STP) v1.0; Requirements Traceability Matrix (RTM, SRS Section 8).

3. Architecture

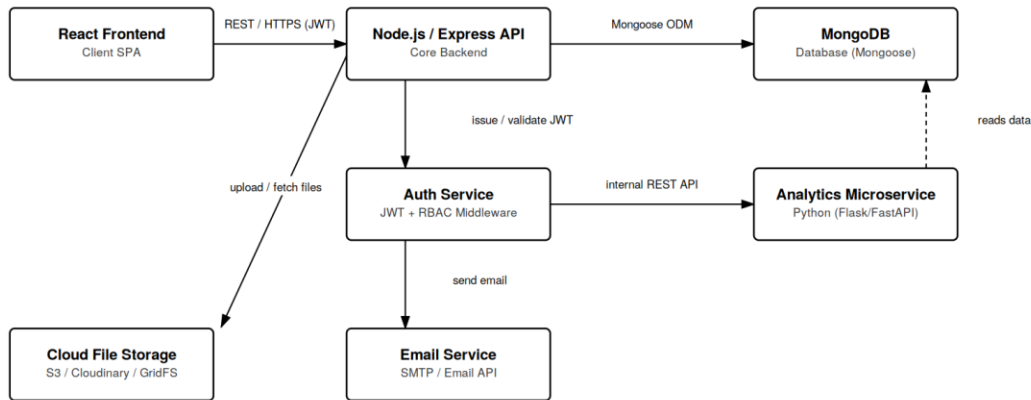
3.1 Goals & Constraints

Goals: a secure, maintainable, and responsive platform supporting 100 concurrent users with 99% availability (CSQP-NF-002, CSQP-NF-003). Constraints: the core application must use the MERN stack and the analytics module must be implemented in Python; only open-source libraries/frameworks may be used; the academic project timeline limits scope; uploaded file size and type restrictions apply.

3.2 Stakeholders & Concerns

Students: usability, responsiveness, data privacy. Instructors: efficient authoring tools, reliable grading, clear analytics. Admins: moderation control, system health visibility. Developers: modularity, maintainability, testability. Course evaluators: compliance with the SRS and measurable quality attributes.

3.3 Component (UML) Diagram



Collaborative Study Material and Quiz Platform – Component Diagram

Figure 1: System Component Diagram

3.4 Component Descriptions

- **React Frontend (Client SPA):** role-specific dashboards and screens for Student, Instructor, and Admin; communicates with the backend over REST/HTTPS.
- **Node.js / Express API (Core Backend):** handles authentication, authorization (RBAC), and the business logic for study material, quizzes, and the question bank; exposes REST endpoints to the frontend.
- **Auth Service:** issues and validates JWT access/refresh tokens and enforces RBAC middleware on protected routes.
- **MongoDB Database:** stores users, material metadata, quizzes, questions, attempts, comments, and notifications, accessed via the Mongoose ODM.
- **Python Analytics Microservice (Flask/FastAPI):** computes individual and class-level performance statistics and serves them to the backend over an internal REST API.
- **Cloud File Storage:** stores uploaded study material files (PDF/DOCX/PPT), referenced by metadata held in MongoDB.
- **Email / Notification Service:** sends password-reset links and in-app/email notifications via SMTP or a third-party email API.

3.5 Chosen Architecture Pattern and Rationale

A layered (N-tier) client-server architecture was chosen for the core application, separating the presentation (React frontend), API/business-logic (Express controllers and services), and data-access (Mongoose models) layers. This gives a clear separation of concerns well suited to the project's academic timeline and four-person team size. A single Python analytics microservice is carved out of this layered core because it has different compute characteristics (statistical aggregation) and a different technology stack to the rest of the system. A full microservices decomposition of every module (auth, material, quiz, question bank) was considered and rejected as unnecessary operational complexity and deployment overhead for a system of this scale.

3.6 Technology Stack & Data Stores

React.js (frontend); Node.js with Express.js (backend REST API); MongoDB with the Mongoose ODM (local instance or MongoDB Atlas); Python with Flask/FastAPI (analytics microservice); JWT for authentication; TLS 1.2+/HTTPS for all communication; cloud object storage or MongoDB GridFS for

uploaded files; an SMTP or third-party email API for notifications; Git/GitHub for version control; GitHub Actions or Jenkins for CI/CD.

3.7 Risks & Mitigations

- **Risk:** Python analytics microservice becomes a bottleneck or single point of failure for reporting. Mitigation: stateless service design that can be restarted or scaled independently of the core API.
- **Risk:** Tight coupling between the frontend and an evolving API contract. Mitigation: a versioned REST API with an OpenAPI/Swagger specification shared across the team.
- **Risk:** MongoDB schema changes breaking multiple modules that share collections. Mitigation: versioned Mongoose schemas and migration scripts reviewed before merge.
- **Risk:** Third-party file-storage or SMTP provider downtime. Mitigation: a retry/queue mechanism for uploads and emails, with provider configuration kept swappable.

3.8 Traceability to Requirements

- CSQP-F-002 (Login with JWT): Auth Service [DS-Auth-01]
- CSQP-F-003 (RBAC enforcement): Auth Service / RBAC Middleware [DS-Auth-02]
- CSQP-F-010 (Upload study material): Material Service [DS-Mat-01]
- CSQP-F-020 (Create quiz): Quiz Service [DS-Quiz-01]
- CSQP-F-022 (Attempt quiz, timed): Quiz Service [DS-Quiz-02]
- CSQP-F-030 (Manage question bank): Question Bank Service [DS-QB-01]
- CSQP-F-040 (Individual performance report): Analytics Microservice [DS-Perf-01]
- CSQP-NF-001 (Response time): Web UI / API [DS-Perf-02]
- CSQP-SR-002 (Password hashing): Auth Service [DS-Sec-01]

3.9 Security Architecture

Threat Modeling (STRIDE):

- **Spoofing:** JWT-based authentication with signed tokens; email verification on registration.
- **Tampering:** server-side input validation and sanitization (CSQP-SR-005); HTTPS/TLS in transit (CSQP-SR-001); write access to MongoDB restricted to the backend's service account.
- **Repudiation:** audit-log entries on material and question edits/deletes (CSQP-F-014), timestamped and tied to the authenticated user.
- **Information Disclosure:** TLS 1.2+ for all traffic (CSQP-SR-001); bcrypt password hashing (CSQP-SR-002); API responses scoped to the caller's role.
- **Denial of Service:** rate limiting on authentication endpoints (CSQP-SR-005); stateless API design allows horizontal scaling.
- **Elevation of Privilege:** RBAC middleware validates the caller's role on every protected route before processing (CSQP-SR-004, CSQP-F-003).

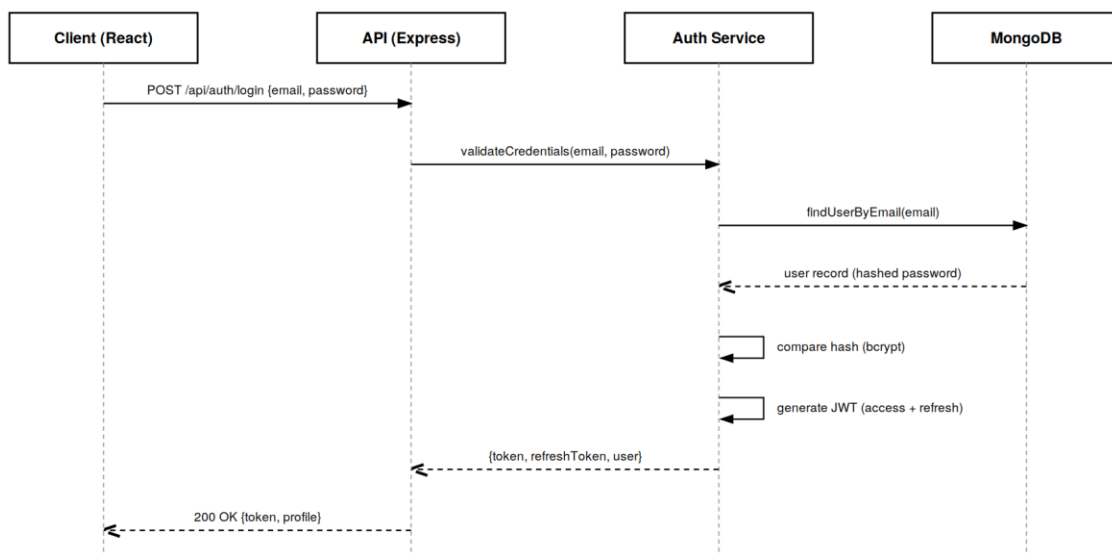
4. Design

4.1 Design Overview

The backend is organized into feature modules (auth, material, quiz, questionBank, analytics-client, collaboration), each following a layered structure: Express route, controller, service (business logic), then Mongoose model (data access). The React frontend mirrors this by feature, with role-specific pages composed from shared UI components. The Python analytics microservice is a separate deployable unit exposing its own REST endpoints, queried by the Node.js backend rather than directly by the frontend.

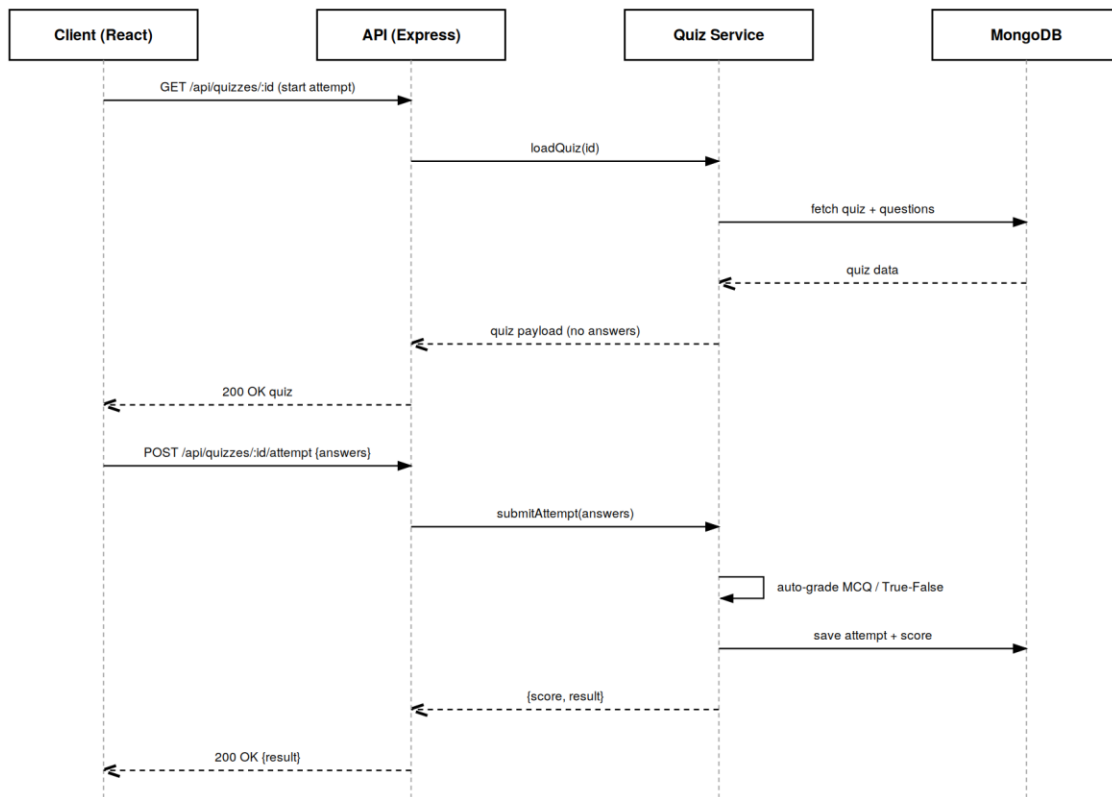
4.2 UML Sequence Diagrams

Two sequence diagrams covering flows specific to this project:



Sequence Diagram 1: User Login & Authentication

Figure 2: Sequence Diagram – User Login & Authentication



Sequence Diagram 2: Quiz Attempt & Auto-Grading

Figure 3: Sequence Diagram – Quiz Attempt & Auto-Grading

4.3 API Design

Interface definitions for three core components:

- **Endpoint:** POST /api/auth/login

Method: POST

Request: {email, password}

Response: {token, refreshToken, user: {id, name, role}}

Errors: 401 Invalid credentials, 429 Too many attempts (rate-limited)

- **Endpoint:** POST /api/quizzes/:quizId/attempt

Method: POST

Request: {answers: [{questionId, response}]}

Response: {status, score, totalQuestions, gradedAt}

Errors: 403 Quiz not assigned to user, 409 Quiz window closed, 422 Invalid answer payload

- **Endpoint:** GET /api/materials?query=&course=&subject=

Method: GET

Request: query parameters (query, course, subject, topic)

Response: {results: [{id, title, course, subject, topic, uploadedBy, downloadCount}], total}

Errors: 400 Invalid query parameters

4.4 Error Handling, Logging & Monitoring

All API errors return a standardized JSON envelope {status, code, message}, produced by a centralized Express error-handling middleware. No passwords, tokens, or other sensitive values are written to logs. Monitoring covers API request latency and error rate, quiz auto-submit/grading failures, and analytics-microservice health checks, with structured logging (e.g., Winston) at configurable log levels.

4.5 UX Design

A responsive React UI with role-specific dashboards (Student / Instructor / Admin); a consistent navigation bar, search bar, and notification icon across screens; consistent form validation and error messaging; and an interface that follows WCAG 2.1 AA guidance for colour contrast and keyboard navigation (CSQP-NF-004), adapting across desktop and mobile breakpoints.

4.6 Open Issues & Next Steps

Candidate future enhancements: integration with an external plagiarism-detection engine, native mobile applications, predictive at-risk-student analytics, and upgrading in-app notifications from polling to a WebSocket-based channel.

5. Appendices

5.1 Glossary

MERN, JWT, RBAC, API, REST, ODM, CI/CD, UI/UX, STRIDE, TLS.

5.2 References

Collaborative Study Material and Quiz Platform SRS v1.0; Software Test Plan (STP) v1.0; IEEE 42010; OWASP Top 10; NIST SP 800-160.

5.3 Tools

draw.io (UML diagrams), Postman (API testing), VS Code, MongoDB Compass, GitHub Actions (CI/CD), Swagger/OpenAPI (API documentation).